

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Bloque 8 · Bootstrap del administrador y validación integral · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. Qué pedía el bloque

El Bloque 8 tuvo dos partes. La primera, el Bloque 7.5.1, cambia cómo se crea el primer administrador: hasta ahora el servidor lo creaba cada vez que arrancaba, y desde este bloque se crea con un script aparte, `npm run seed:admin`, que solo se ejecuta cuando alguien lo decide. La segunda es el Laboratorio 8, que valida la API completa con tres matrices de pruebas:

- **P01 a P36:** las pruebas del Bloque 7.5, con usuarios, perfiles, recursos, estados, BOLA, API Key y persistencia.
- **SEG-01 a SEG-08:** comprobaciones complementarias de lo que puede ver cada rol.
- **BA01 a BA09:** las pruebas del bootstrap. BA08 y BA09 solo se pueden hacer en una base de pruebas aislada, nunca en la de trabajo.

Una prueba solo se aprueba cuando coinciden el código HTTP, el cuerpo de la respuesta y el estado de los datos. Los resultados posibles son APROBADA, FALLIDA, BLOQUEADA o PENDIENTE, y si algo falla se abre una incidencia en lugar de cambiar el resultado esperado.

El resultado, en una línea

Las 53 pruebas quedaron APROBADAS, no se abrió ninguna incidencia y el bootstrap pasó de cumplir 4 a cumplir las 10 reglas del Bloque 7.5.1.

2. El problema del bootstrap anterior

Antes de cambiar nada comparamos nuestro bootstrap con el del profesor. La diferencia era una sola, pero importante: el nuestro preguntaba si existía un usuario **con ese correo**, y el del profesor pregunta si existe **algún administrador**.

Lo comprobamos contra la base en lugar de suponerlo. Con el mismo `ADMIN_EMAIL`, arrancar dos veces dejaba un solo administrador. Pero al cambiar el correo en el `.env` y arrancar otra vez, la base quedaba con **dos** administradores. Como el bootstrap corría en cada `npm start`, bastaba con editar el `.env` y reiniciar para fabricar una cuenta administradora nueva.

De las diez reglas del README 7.5.1, el teatro cumplía cuatro. Le faltaban:

- validar el largo de la contraseña (mínimo 12 caracteres) y el formato del correo;
- trabajar dentro de una transacción con bloqueo, para que dos ejecuciones a la vez no choquen;

- no hacer nada si ya existe cualquier administrador, aunque esté inactivo;
- negarse si el correo ya pertenece a otra cuenta, en lugar de arriesgar una elevación de privilegios;
- cerrar con `COMMIT` o `ROLLBACK` y cerrar el pool de conexiones al terminar.

Antes: app.js creaba el administrador al arrancar — Bloque 7

Archivo: src/app.js (commit 1f3fcd8)

```
177  // Arranque del servidor
178  // Primero el administrador inicial, después se abre el puerto
179  // ======================================
180  // El orden importa: si el servidor atendiera peticiones antes de sembrar el
181  // administrador, habría una ventana en la que las operaciones administrativas
182  // no tendrían a nadie que pudiera ejecutarlas. Por eso el listen va dentro del
183  // then, no al lado.
184  usuariosService
185    .crearAdministradorInicial()
186    .then(() => {
187      app.listen(PUERTO, () => {
188        console.log(`Servidor escuchando en http://localhost:${PUERTO}`);
189        console.log(`Swagger UI: http://localhost:${PUERTO}/api-docs`);
190        console.log(`OpenAPI JSON: http://localhost:${PUERTO}/openapi.json`);
191      });
192    })
193    .catch((error) => {
194      console.error("No se pudo preparar el administrador inicial:", error);
195      process.exit(1);
196    });
```

> Cada npm start o npm run dev ejecutaba el bootstrap antes de abrir el puerto.

API_TEATRO · Bloque 8 · Bootstrap y validación integral

Figura 1. Antes: app.js ejecutaba el bootstrap en cada arranque, antes de abrir el puerto.

3. El bootstrap nuevo

Separamos dos cosas que no tienen por qué ir juntas: **arrancar el servicio** y **entregar la primera llave maestra**. La primera pasa muchas veces; la segunda, una sola vez, al instalar.

Ahora: app.js solo abre el puerto — Bloque 8

Archivo: src/app.js

```
175  // Arranque del servidor
176  // Solo abre el puerto: no crea ni toca ninguna cuenta
177  // ======================================
178  app.listen(PUERTO, () => {
179    console.log(`Servidor escuchando en http://localhost:${PUERTO}`);
180    console.log(`Swagger UI: http://localhost:${PUERTO}/api-docs`);
181    console.log(`OpenAPI JSON: http://localhost:${PUERTO}/openapi.json`);
182  });
183
184  module.exports = app;
```

> El servidor ya no crea ni toca ninguna cuenta. El primer administrador se crea aparte, con npm run seed:admin.

API_TEATRO · Bloque 8 · Bootstrap y validación integral

Figura 2. Ahora: app.js solo abre el puerto y no crea ni toca ninguna cuenta.

Reescribimos `crearAdministradorInicial()` con las diez reglas. El hash de la contraseña se calcula antes de abrir la transacción, porque bcrypt tarda y no queremos tener un bloqueo abierto mientras tanto. Después la función toma el bloqueo `pg_advisory_xact_lock(7505, 1)`, revisa si existe cualquier administrador y solo si no existe, y si el correo está libre, inserta la cuenta.

El bootstrap nuevo: crearAdministradorInicial() — transacción, bloqueo asesor y sin elevación

Archivo: src/services/usuarios.service.js

```
185   // El hash se calcula fuera de la transacción: bcrypt tarda, y mantener un
186   // bloqueo abierto mientras tanto retrasaría a cualquier otra ejecución.
187   const passwordHash = await generarPasswordHash(password);
188
189   const cliente = await pool.connect();
190   let enTransaccionAbierta = false;
191
192   try {
193     await cliente.query("BEGIN");
194     enTransaccionAbierta = true;
195
196     await cliente.query(
197       "SELECT pg_advisory_xact_lock($1, $2)",
198       CLAVE_BLOQUEO_BOOTSTRAP
199     );
    …
201     // Cualquier administrador cuenta, aunque esté inactivo: reactivarlo es una
202     // operación administrativa, no trabajo del bootstrap.
203     const administradores = await cliente.query(
204       `SELECT u.id
205        FROM public.usuarios u
206        INNER JOIN public.roles r ON r.id = u.rol_id
207        WHERE r.nombre = 'administrador'
208        LIMIT 1`
209     );
210
211     if (administradores.rows.length > 0) {
212       await cliente.query("COMMIT");
213       enTransaccionAbierta = false;
214
215       return { creado: false, motivo: "ADMINISTRADOR_EXISTENTE" };
216     }
    …
218     // Si el correo ya es de otra cuenta, el bootstrap no la convierte en
219     // administradora: eso sería una elevación de privilegios por el .env.
220     const correoOcupado = await cliente.query(
221       `SELECT id FROM public.usuarios WHERE LOWER(email) = LOWER($1) LIMIT 1`,
222       [email]
223     );
224
225     if (correoOcupado.rows.length > 0) {
226       throw new Error(
227         "El correo del bootstrap ya pertenece a otra cuenta. No se elevarán privilegios."
228       );
229     }
    …
249     await cliente.query("COMMIT");
250     enTransaccionAbierta = false;
251
252     return {
253       creado: true,
254       motivo: "ADMINISTRADOR_CREADO",
255       usuario: creado.rows[0]
256     };
257   } catch (error) {
258     if (enTransaccionAbierta) await cliente.query("ROLLBACK");
259     throw error;
260   } finally {
261     cliente.release();
262   }
```

La comprobación es por rol, no por correo: si existe cualquier administrador, aunque esté inactivo, no hace nada. Si el correo ya es de otra cuenta, se niega en lugar de convertirla en administradora.

API_TEATRO · Bloque 8 · Bootstrap y validación integral

Figura 3. crearAdministradorInicial(): transacción, bloqueo asesor, comprobación por rol y rechazo de la elevación.

Alrededor de esa función quedaron tres piezas nuevas:

- **scripts/crear-admin.js**, que se ejecuta con `npm run seed:admin`, informa si creó o no la cuenta, sale con código 1 ante cualquier error y cierra el pool con la nueva función

`cerrarPool()`. Sin ella, el script terminaría su trabajo y el proceso se quedaría esperando consultas que no llegan.

- **pruebas/correr.sh**, el lanzador de las baterías. Como ahora el servidor no crea el administrador, cada batería sigue el flujo: reiniciar la base, `npm run seed:admin`, arrancar y probar. El seed queda en las pruebas y no dentro de `npm start`.
- **scripts/diagnostico-rls.js**, adaptado del script del profesor, que muestra con qué rol se conecta la API y si ese rol pasa por las políticas de RLS (sección 9).

Con el cambio, las nueve baterías siguieron pasando: 356 de 356.

4. Cómo hicimos el Laboratorio 8

Montamos el escenario que pide la sección 4 del README: cuatro identidades con su propio token (administrador, asistente A, asistente B y una cuenta de taquilla), y una boleta para cada asistente. Los ids no se dieron por supuestos: se leyeron de la base después de montar el escenario.



Figura 4. Escenario del laboratorio con los ids reales de la base.

Las matrices del README las adaptamos a los recursos del teatro, como en los laboratorios anteriores: asistentes, funciones, localidades y boletas. La taquilla es solo un rol: no tiene perfil ni boletas propias. Los ids P01 a P36 y SEG-01 a SEG-08 se conservaron sin renumerar, y el resultado esperado de cada fila salió del comportamiento real del teatro.

Ninguna prueba se dio por aprobada con evidencias de bloques anteriores: P34 y P36, que ya habíamos probado antes, se ejecutaron de nuevo. En las capturas las contraseñas se ven como `••••••••`, los tokens recortados como `eyJhbGci…` y no aparecen ni la API Key, ni el host, ni las cadenas de conexión.

5. Matriz P01 a P36

8.1 Usuarios y autenticación

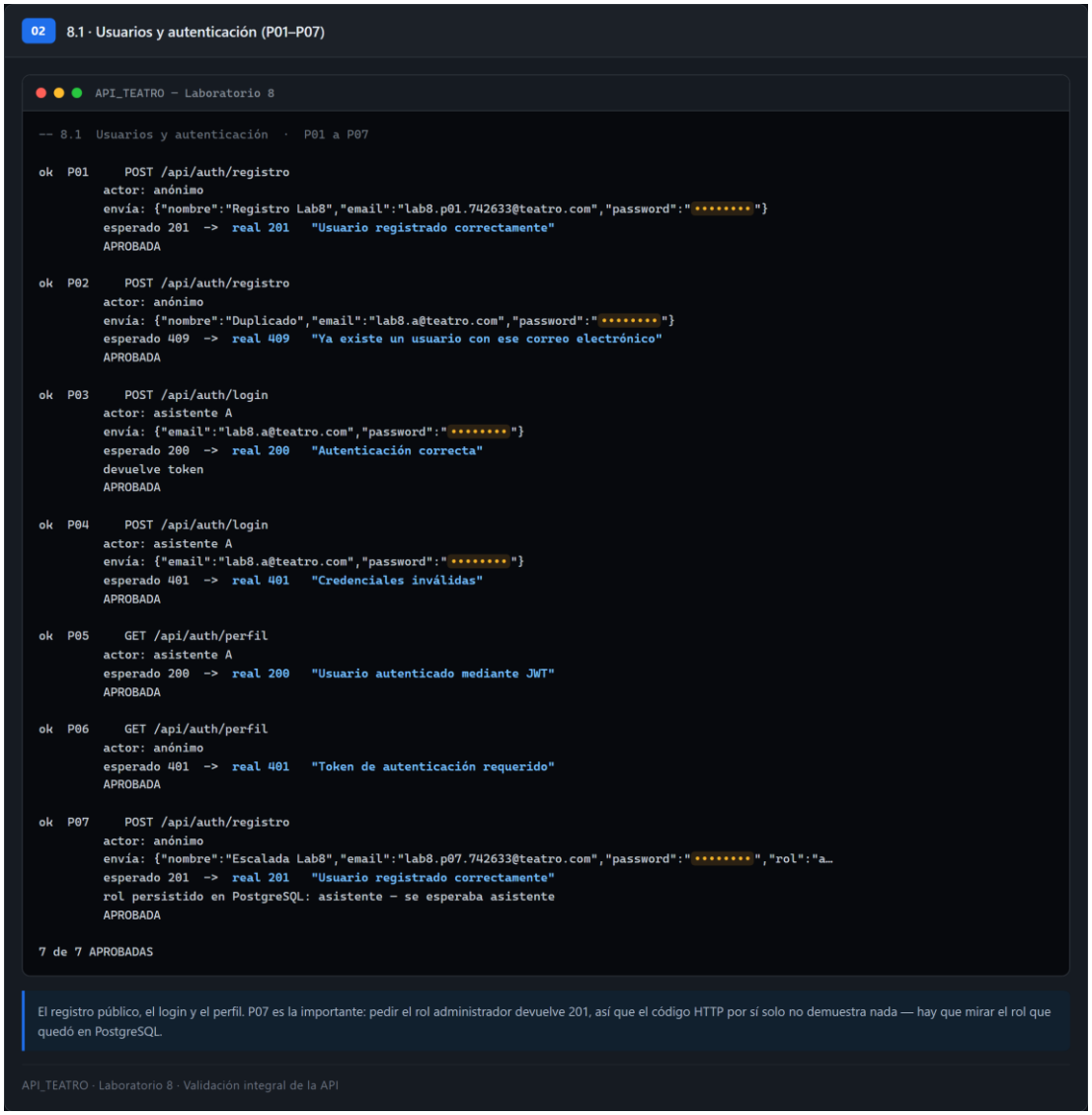


Figura 5. P01 a P07: registro, correo duplicado, login, contraseña incorrecta y perfil con y sin JWT.

P07 es la prueba más engañosa del laboratorio. El registro público responde 201 aunque el cuerpo pida `rol: "administrador"`, así que mirando solo el código parecería que el ataque funcionó o que no pasó nada. Lo que demuestra el resultado es la consulta a PostgreSQL: la cuenta quedó como asistente, porque el validador no declara el campo `rol` y `matchedData` lo descarta.

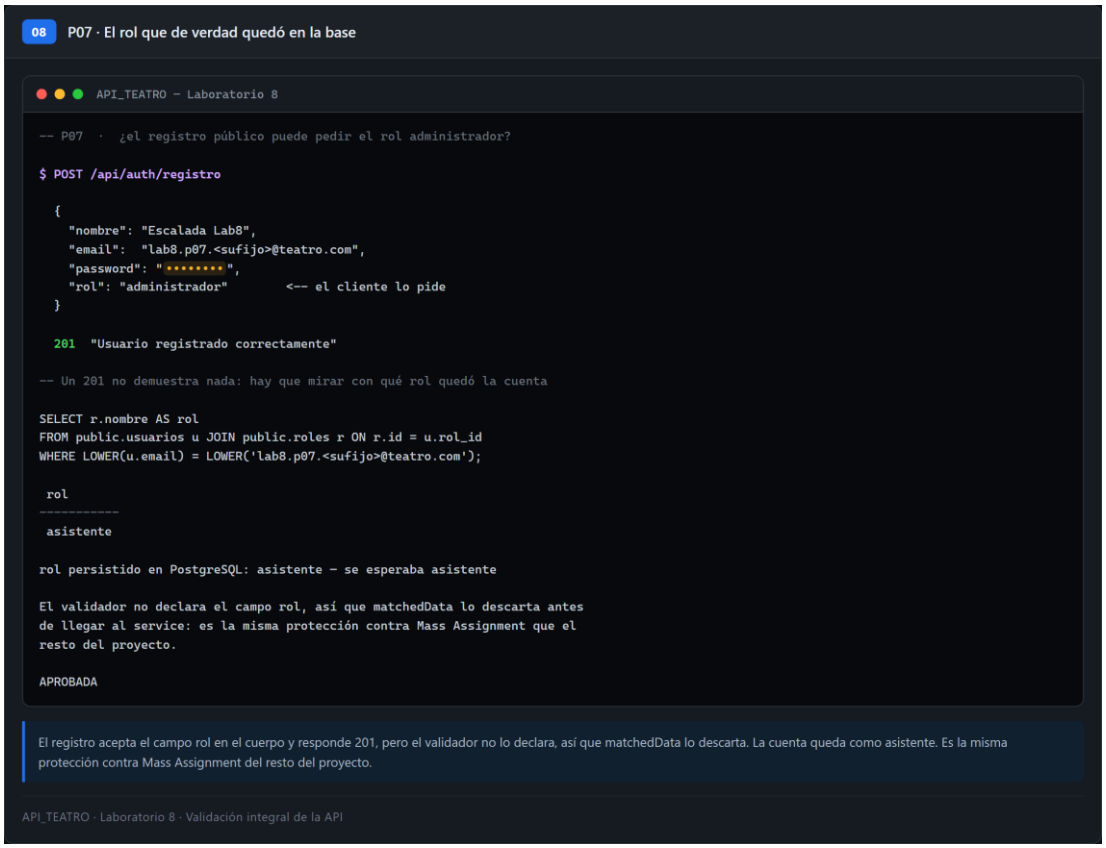


Figura 6. P07: el registro responde 201, pero la consulta SQL muestra que la cuenta quedó como asistente.

8.2 Asistentes

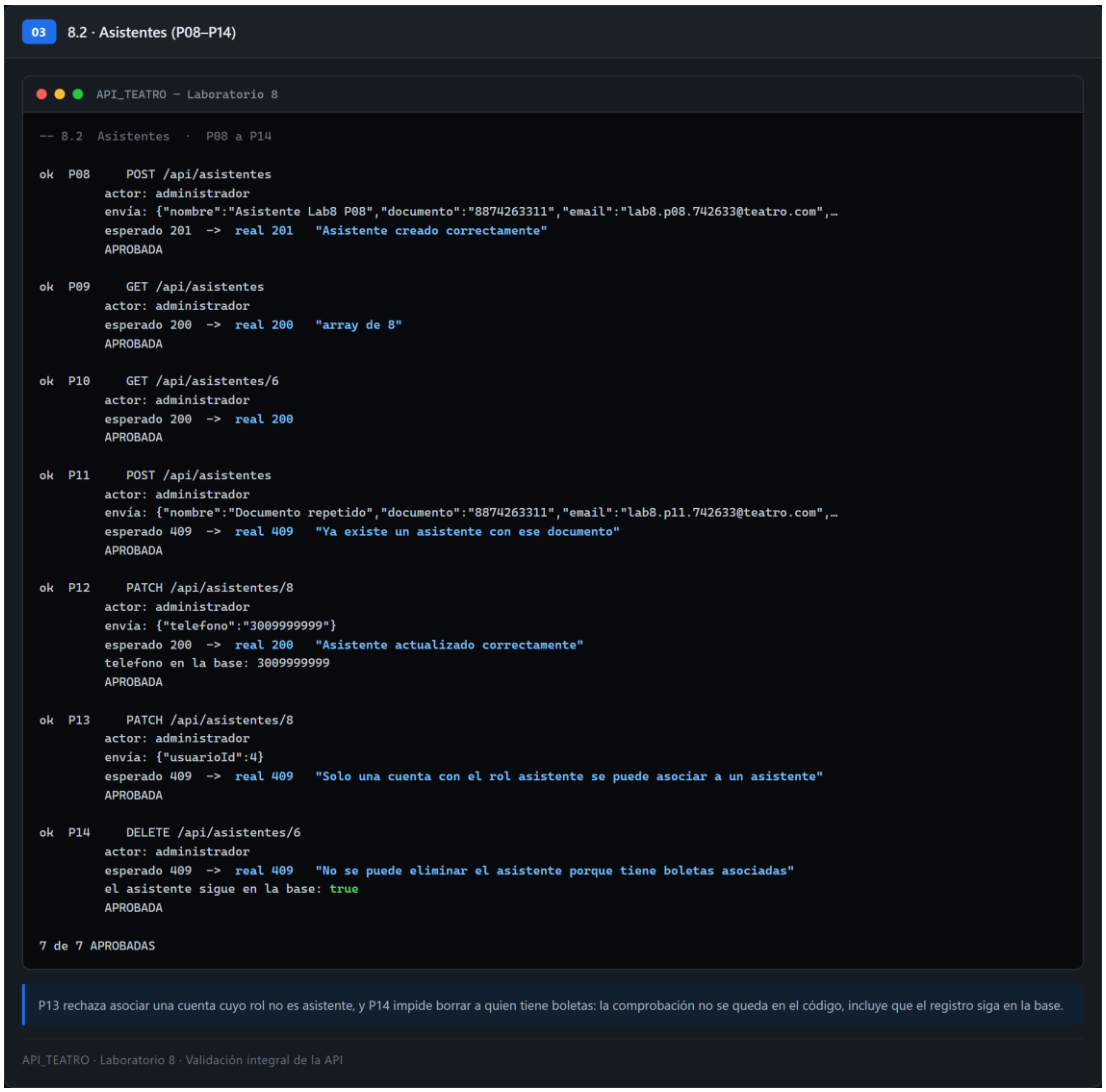


Figura 7. P08 a P14: crear, listar, consultar, documento duplicado, PATCH, asociación con otro rol y borrado con boletas.

8.3 Funciones y cuentas administrativas

Este bloque de la guía trata un recurso con perfil propio. En el teatro lo hicimos con las funciones, que tienen la misma forma: se crean sobre un evento activo, tienen una regla de unicidad (la sala única), se listan por evento y no se pueden borrar si tienen boletas. P18 y P19 pasaron a la creación de cuentas de taquilla, que es el mecanismo equivalente del teatro.

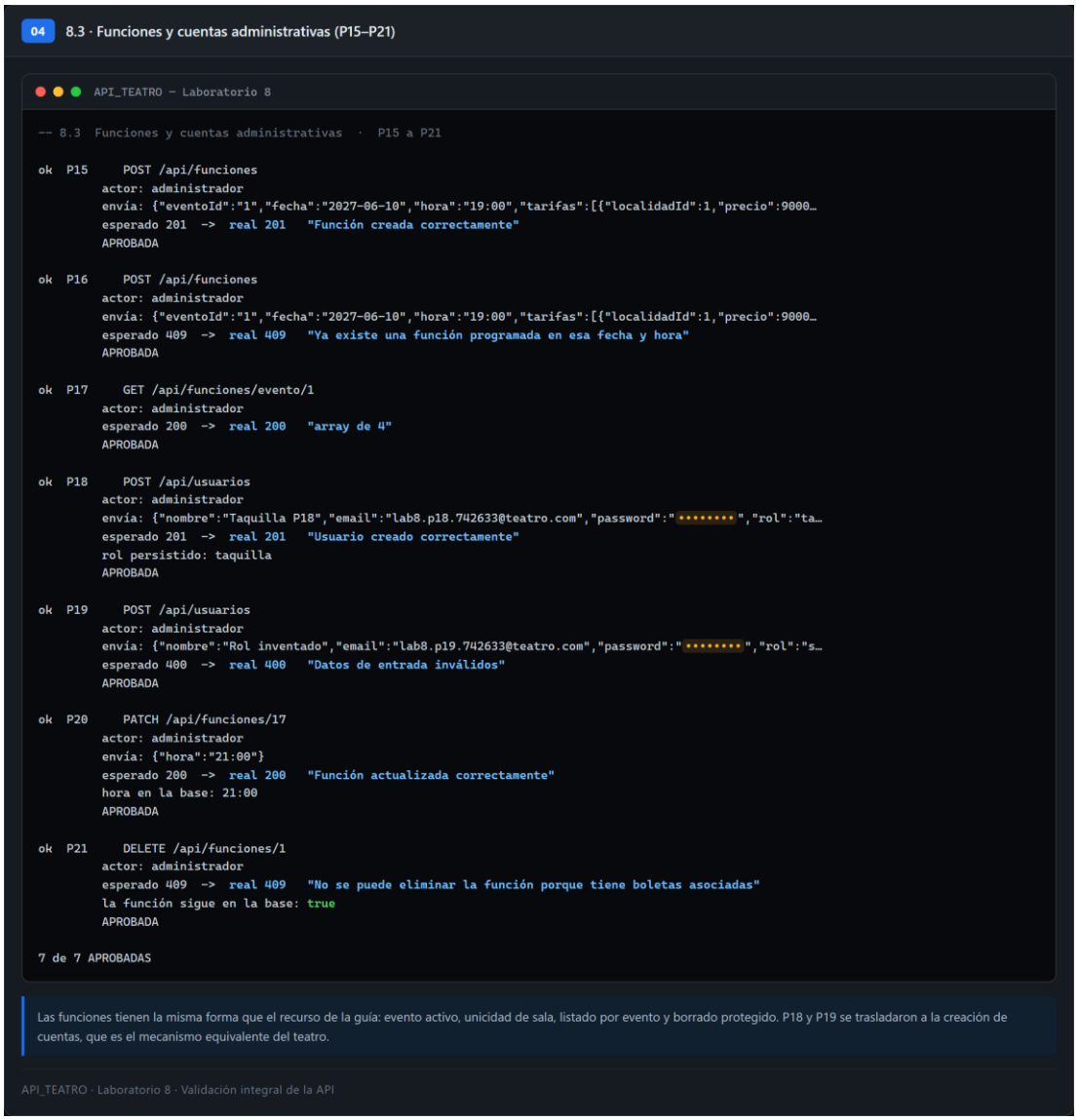


Figura 8. P15 a P21: funciones, sala única, listado por evento y cuentas de taquilla creadas por el administrador.

8.4 Localidades

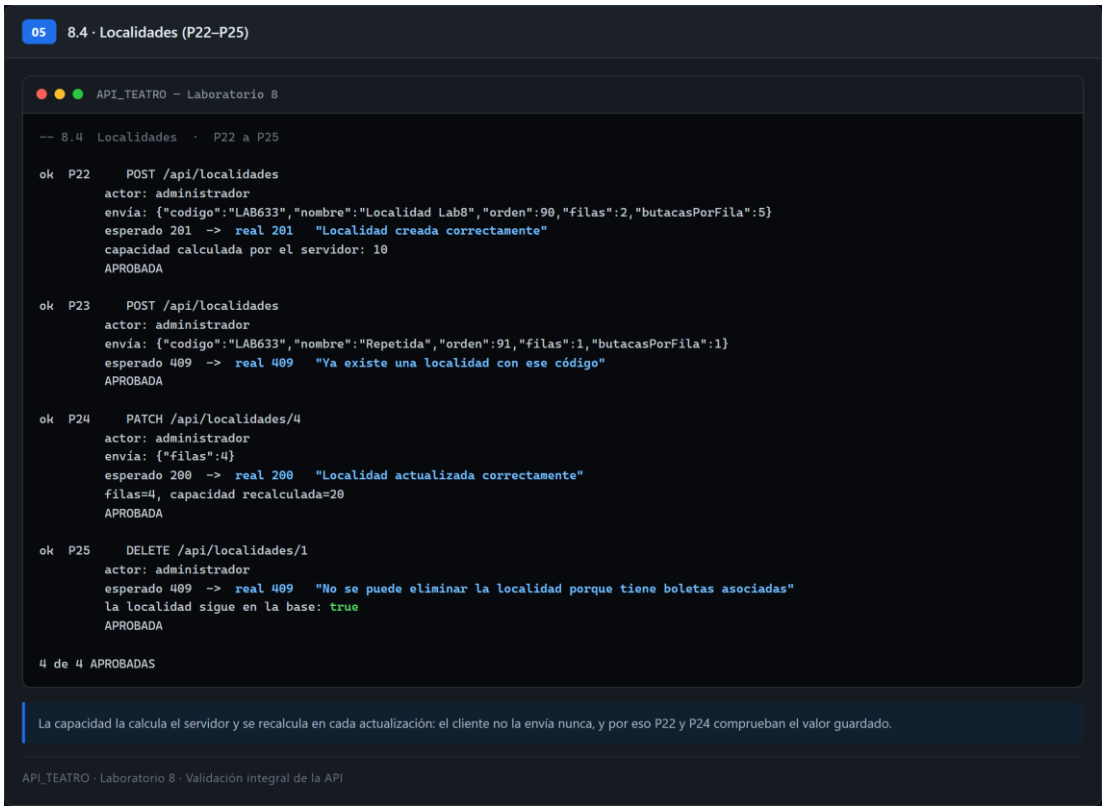


Figura 9. P22 a P25: crear, código duplicado, PATCH con capacidad recalculada y borrado con boletas.

8.5 Boletas



Figura 10. P26 a P33: compra, función inexistente o agotada, butaca ocupada o fuera de aforo y la máquina de estados.

8.6 BOLA y API Key

En P34 el asistente B pide la boleta del asistente A con una API Key válida. El README insiste en esto: un 403 por falta de API Key no demostraría BOLA. Además revisamos que el cuerpo no trajera datos de la boleta ajena.



Figura 11. P34: boleta ajena con API Key válida, 403 sin filtrar datos. P35: sin X-API-Key, 401.

8.7 Persistencia

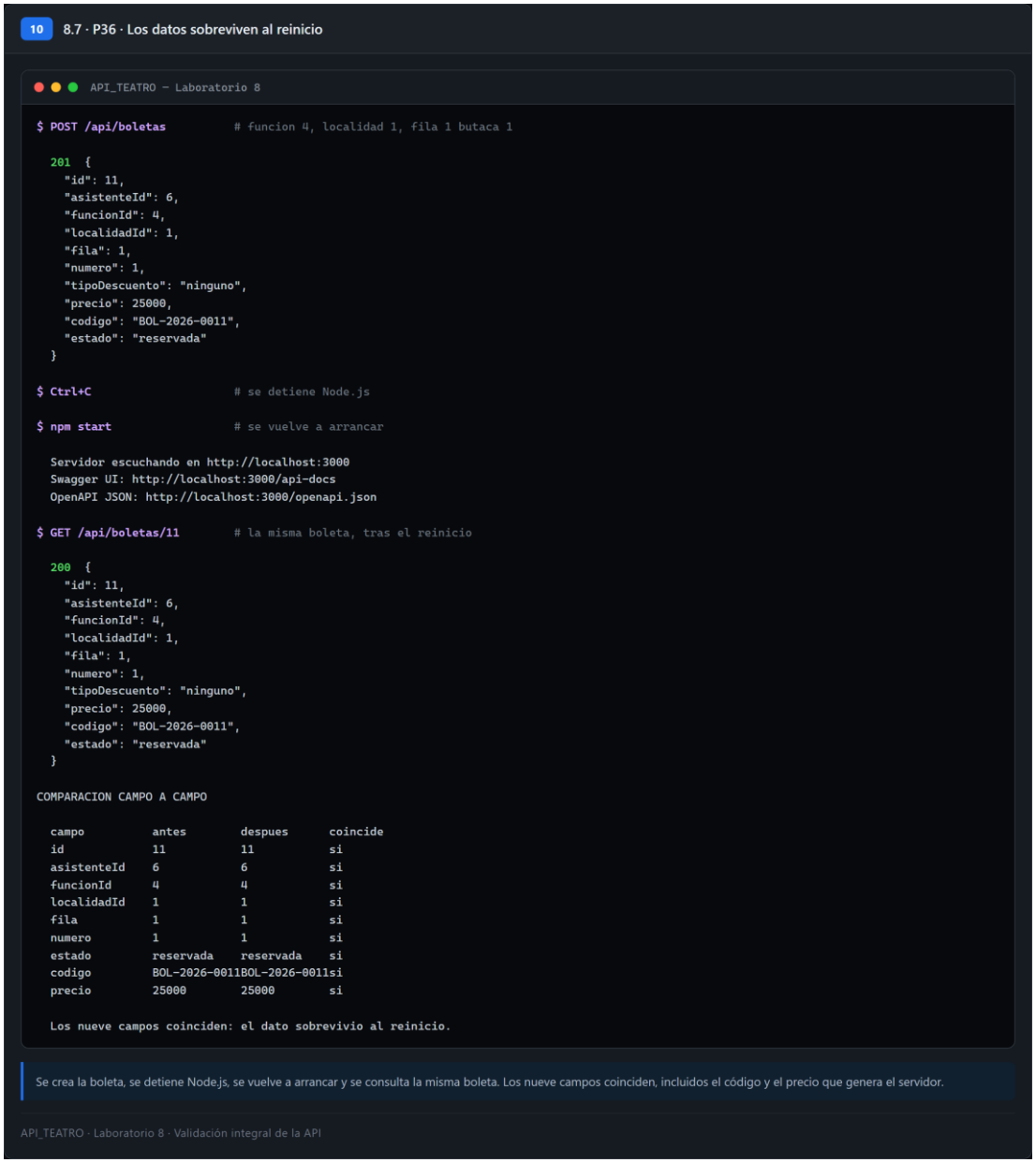


Figura 12. P36: se crea la boleta, se detiene Node, se vuelve a arrancar y los nueve campos coinciden.

Resumen de la matriz:

ID	Prueba en el teatro	Esperado	Real	Estado
P01	Registro público válido	201	201	APROBADA
P02	Correo duplicado	409	409	APROBADA
P03	Login correcto	200	200	APROBADA
P04	Contraseña incorrecta	401	401	APROBADA
P05	Perfil con JWT válido	200	200	APROBADA
P06	Perfil sin JWT	401	401	APROBADA
P07	Pedir rol administrador en el registro	201, asistente	201, asistente	APROBADA
P08	Crear asistente	201	201	APROBADA
P09	Listar asistentes	200	200	APROBADA
P10	Consultar asistente por id	200	200	APROBADA

ID	Prueba en el teatro	Esperado	Real	Estado
P11	Documento duplicado	409	409	APROBADA
P12	PATCH de asistente	200	200	APROBADA
P13	Asociar cuenta con otro rol	409	409	APROBADA
P14	Borrar asistente con boletas	409	409	APROBADA
P15	Crear función con evento activo	201	201	APROBADA
P16	Sala única: misma fecha y hora	409	409	APROBADA
P17	Listar funciones por evento	200	200	APROBADA
P18	Administrador crea cuenta de taquilla	201	201	APROBADA
P19	Crear cuenta con rol no permitido	400	400	APROBADA
P20	PATCH de función	200	200	APROBADA
P21	Borrar función con boletas	409	409	APROBADA
P22	Crear localidad	201	201	APROBADA
P23	Código de localidad duplicado	409	409	APROBADA
P24	PATCH de localidad	200	200	APROBADA
P25	Borrar localidad con boletas	409	409	APROBADA
P26	Comprar boleta válida	201	201	APROBADA
P27	Función inexistente	400	400	APROBADA
P28	Función agotada	409	409	APROBADA
P29	Butaca ocupada	409	409	APROBADA
P30	Butaca fuera del aforo	409	409	APROBADA
P31	reservada → pagada	200	200	APROBADA
P32	pagada → usada	200	200	APROBADA
P33	usada → reservada	409	409	APROBADA
P34	Asistente consulta boleta ajena	403	403	APROBADA
P35	Sin X-API-Key	401	401	APROBADA
P36	Reiniciar y volver a consultar	200, mismos datos	200, mismos datos	APROBADA

Lo que hicimos distinto a la guía

- **P15 a P21:** el teatro no tiene una tabla de perfiles de personal, así que se probaron sobre las funciones, que tienen la misma forma.
- **P19 responde 400, no el 409 de la guía:** el rol de la cuenta se revisa con una lista blanca en el validador, antes de llegar al service. Es una diferencia de capa, no de criterio.
- **P27 a P30:** las reglas de la guía sobre un recurso inexistente, inactivo u ocupado se traducen a función inexistente, función agotada, butaca ocupada y butaca fuera del aforo.
- **P35 responde 401, no 403:** el README pide verificar el código configurado. Nuestro middleware responde 401 cuando falta la API Key o es inválida, y 403 solo cuando está deshabilitada.

6. Matriz complementaria de seguridad

En la guía, SEG-04 a SEG-07 parten de un rol que tiene su propia agenda. En el teatro no hay agenda de taquilla, así que no inventamos un 403 sobre algo que no existe: comprobamos los límites reales de ese rol. La taquilla no tiene boletas propias, no ve la boletería completa y no puede hacer operaciones administrativas. En cada respuesta 200 revisamos también el cuerpo, para detectar datos ajenos aunque el código pareciera correcto.



Figura 13. SEG-01 a SEG-08: lo que puede y no puede ver cada rol, revisando también el cuerpo de las respuestas.

ID	Prueba en el teatro	Esperado	Real	Estado
SEG-01	Asistente A consulta su boleta	200	200	APROBADA
SEG-02	Asistente A consulta la boleta de B	403	403	APROBADA

ID	Prueba en el teatro	Esperado	Real	Estado
SEG-03	Asistente A consulta /mias	200, solo suyas	200, 2 propias	APROBADA
SEG-04	Taquilla consulta /mias	403	403	APROBADA
SEG-05	Asistente A consulta las boletas de B	403	403	APROBADA
SEG-06	Taquilla lista toda la boletería	403	403	APROBADA
SEG-07	Taquilla intenta crear un evento	403	403	APROBADA
SEG-08	Administrador consulta una boleta	200	200	APROBADA

7. Matriz del bootstrap

En la base de trabajo

Corrimos el bootstrap dos veces seguidas y consultamos los administradores antes y después con la consulta del README. Las dos ejecuciones respondieron que ya existía una cuenta administradora y no cambiaron nada.

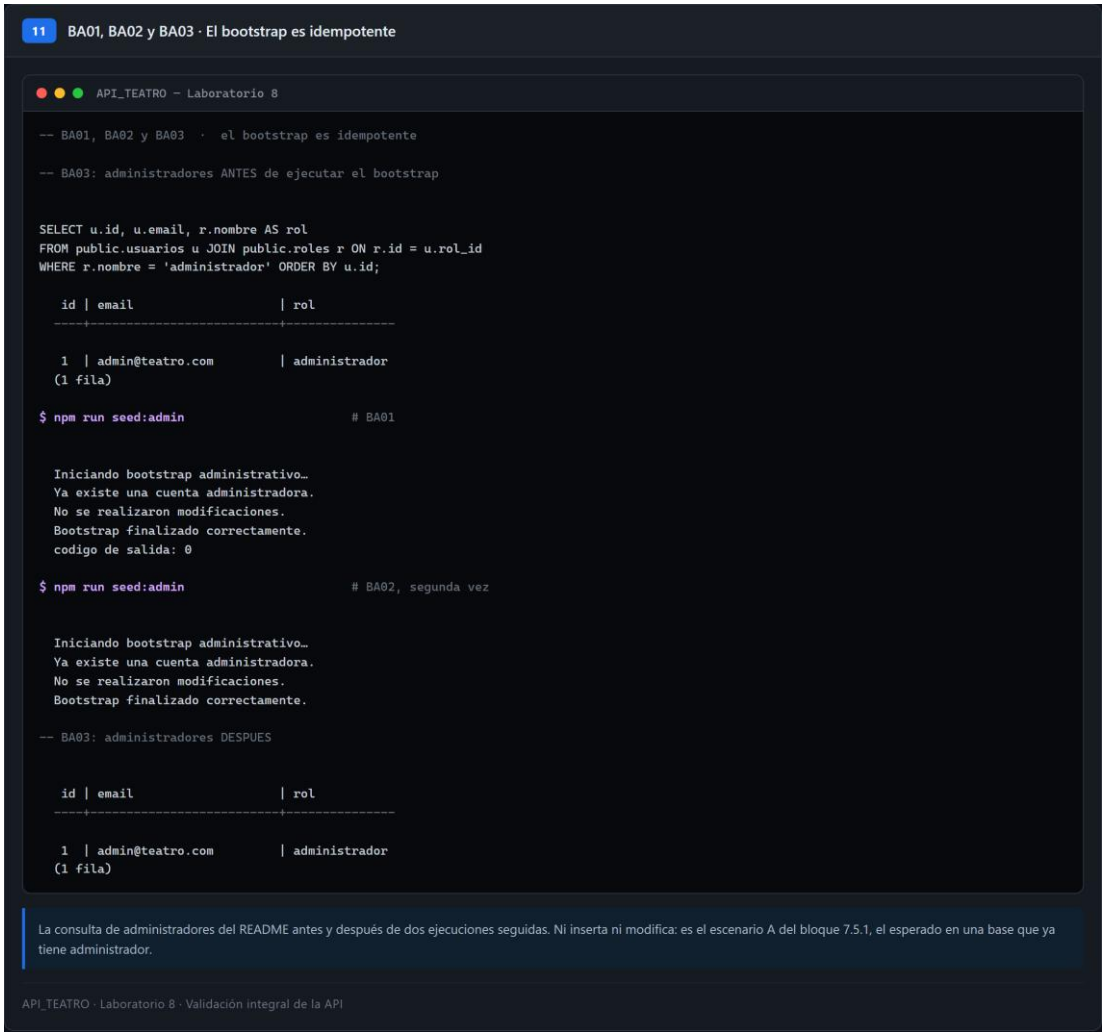


Figura 14. BA01 a BA03: dos ejecuciones con administrador existente; antes y después, el mismo administrador.

Después arrancamos la API sin las variables `ADMIN_*` y funcionó normalmente, porque ya no las necesita. El script, en cambio, se negó a correr sin ellas y salió con código 1, que es lo que permite detectarlo en una instalación. El administrador existente siguió entrando con su contraseña de siempre: el bootstrap no la cambió.

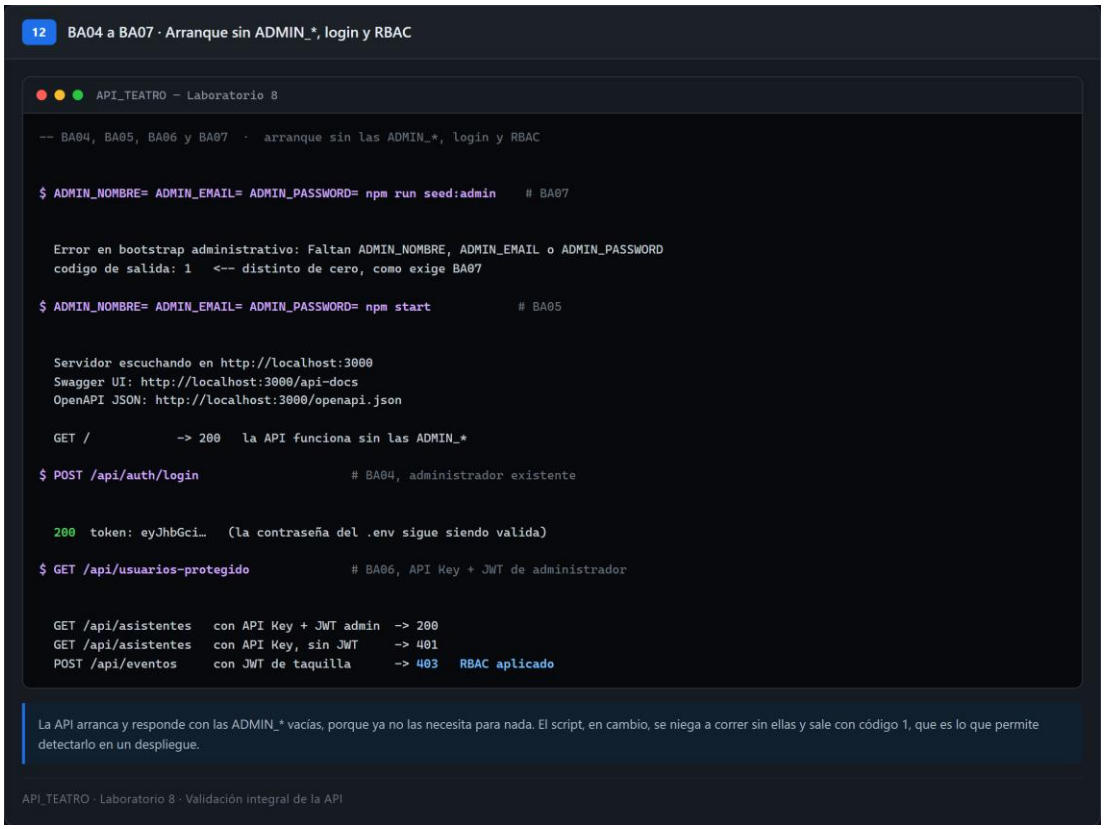


Figura 15. BA04 a BA07: login del administrador, arranque sin ADMIN_*, RBAC y el script sin variables.

En una base aislada

BA08 y BA09 necesitan una base sin administradores, y el README prohíbe vaciar la de trabajo para simularlo. Por eso las hicimos en un proyecto de Supabase independiente del de trabajo. Allí montamos el esquema, RLS y los datos semilla (scripts 01, 02 y 04); no hacía falta el rol `api_teatro`, porque estas pruebas revisan la lógica del bootstrap y no RLS.

Antes de tocar nada comprobamos que el host del proyecto de pruebas era distinto del de trabajo, y el script lleva una salvaguarda: si las dos cadenas apuntaran al mismo proyecto, se detiene antes de ejecutar. La conexión también usó TLS verificado con el certificado de Supabase.

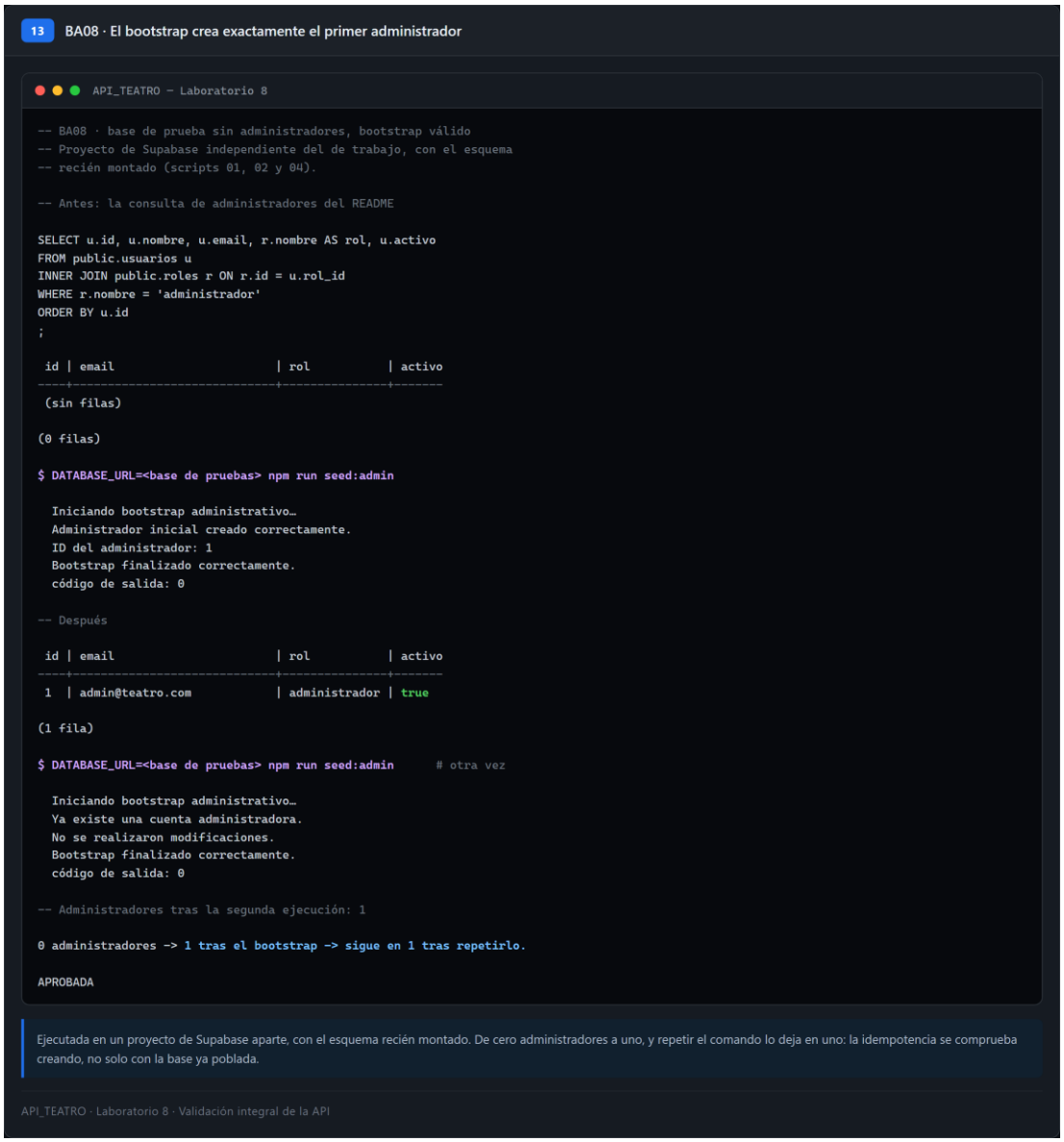


Figura 16. BA08: en la base de pruebas, de 0 administradores a 1, y repetir el comando lo deja en 1.

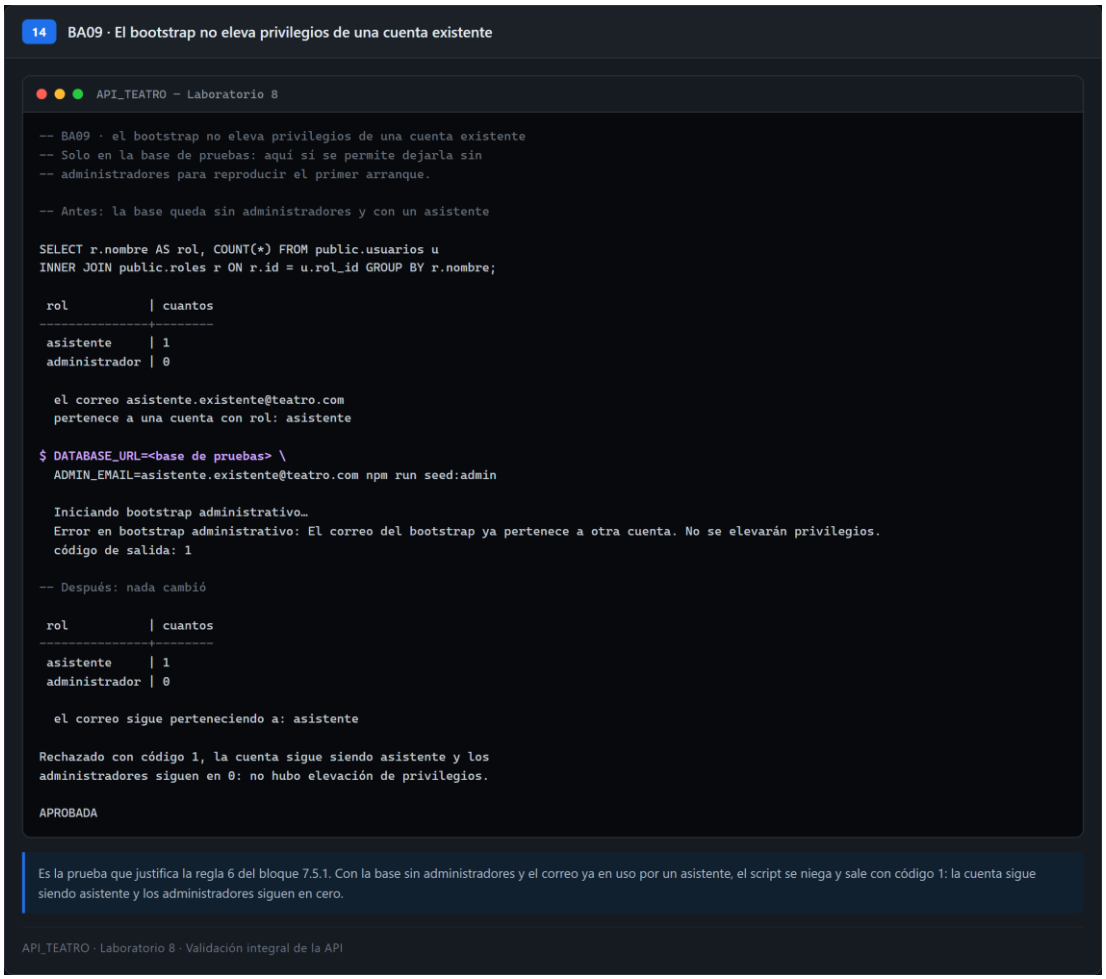


Figura 17. BA09: con el correo de un asistente, el bootstrap se niega con código 1 y no eleva privilegios.

ID	Escenario	Esperado	Real	Estado
BA01	Bootstrap con administrador existente	No modifica nada	Sin cambios	APROBADA
BA02	Bootstrap por segunda vez	No duplica	Sigue 1	APROBADA
BA03	Administradores antes y después	Los mismos	Los mismos	APROBADA
BA04	Login del administrador existente	200 y JWT	200 y JWT	APROBADA
BA05	API sin las ADMIN_*	Arranca	Arranca	APROBADA
BA06	Recurso protegido con API Key y JWT	Según RBAC	200 / 401 / 403	APROBADA
BA07	seed:admin sin las ADMIN_*	Error, salida ≠ 0	Salida 1	APROBADA
BA08	Base vacía, bootstrap válido	Crea 1	0 → 1 → 1	APROBADA
BA09	Base vacía, correo de un asistente	Rechaza	Salida 1, 0 admins	APROBADA

8. Incidencias

Ninguna prueba resultó FALLIDA, así que no se abrió ninguna incidencia y no hubo correcciones de código durante la ejecución. BA08 y BA09 estuvieron BLOQUEADAS mientras no tuvimos la base aislada: una prueba bloqueada no es una incidencia, es una

precondición que falta, y se ejecutaron en cuanto la tuvimos. El registro quedó en `docs/seguridad/informe_incidencias.md`.

Sí aprendimos algo de nuestras propias evidencias: en la primera versión de las capturas, el cuerpo de P02 y P03 mostraba la contraseña de las cuentas de prueba. Las demás se habían salvado solo porque el texto se cortaba por largo, lo cual no es una protección. Tapamos la contraseña desde el origen y volvimos a generar todas las capturas.

9. Diagnóstico de Row Level Security

El material del profesor incluye un script para ver con qué rol se conecta la API y si ese rol se salta RLS. En el teatro, gracias al rol `api_teatro` del Bloque 7, no se lo salta: no es superusuario, no puede saltarse RLS, no es dueño de ninguna tabla y cada tabla tiene su política.

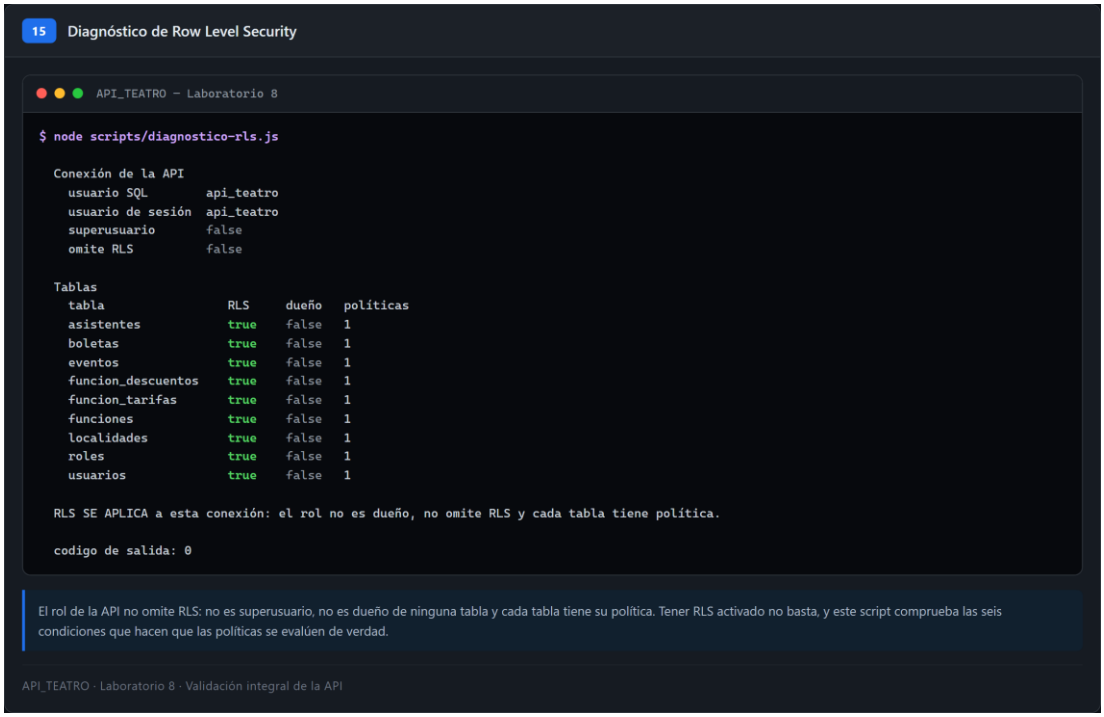


Figura 18. scripts/diagnostico-rls.js: la conexión de la API no omite RLS y las 9 tablas tienen política.

10. Semgrep y npm audit

Semgrep revisó 78 archivos con 76 reglas y no encontró nada, incluidos los archivos nuevos del bootstrap. Como el ejecutable de Semgrep sigue bloqueado por el Control inteligente de aplicaciones de Windows, lo corrimos con Docker, sin desactivar esa protección.

`npm audit`, en cambio, sí encontró algo, aunque no habíamos cambiado dependencias: se habían publicado avisos nuevos. Había tres vulnerabilidades. La importante era de

severidad alta en `body-parser`, que llega por Express y es justo lo que lee el cuerpo de todas las peticiones de la API (una denegación de servicio con un cuerpo comprimido). Las otras dos eran moderadas, en `fast-uri` y `ip-address`. Las corregimos con `npm audit fix`, sin `--force`, así que solo se actualizaron versiones dentro de los rangos que ya teníamos y `package.json` no cambió. Quedaron 0 vulnerabilidades en 173 paquetes, con y sin las dependencias de desarrollo, y las 356 pruebas siguieron pasando.

11. Lo que este bloque no resuelve

- **No hay auditoría.** El script dice en pantalla qué hizo, pero en la base no queda registro de quién creó la primera cuenta ni cuándo.
- **El bootstrap no recupera accesos.** No reinicia contraseñas ni reactiva cuentas. Si se pierde el acceso administrativo hace falta otro procedimiento, que todavía no existe.
- **No hay superadministrador.** Un administrador puede crear otros administradores desde la API. Separar un rol por encima haría esa concesión más restrictiva.
- **Las variables ADMIN siguen en el .env de desarrollo.** Las pruebas las usan para iniciar sesión. En un despliegue real habría que retirarlas después de crear la cuenta; BA05 y BA07 comprueban que la API funciona sin ellas y que el script se niega a correr sin ellas.
- **El bloqueo asesor solo coordina este script.** No protege otras rutas que creen administradores; esas dependen de su propia autorización.

12. Repositorio

https://github.com/JuanK2550/API-TEATRO

El bloque quedó en cinco commits: el bootstrap con las diez reglas, `cerrarPool()` y `app.js` sin el bootstrap en `d8841b5`; el script `seed:admin`, el diagnóstico de RLS y el lanzador de pruebas en `c0219f3`; la actualización de dependencias en `6de0a4b`; y el informe técnico (`docs/seguridad/informe-bloque8.md`), el registro de incidencias, los análisis de Semgrep y npm audit, el README y el `.env.example` en `4c3206e`. El último ajuste al informe técnico quedó en `85d33e2`.